

LLMs on XR (LoXR): Performance Evaluation of LLMs Executed Locally on Extended Reality Devices

Xinyu Liu^{1,2}, Dawar Khan^{1*}, Omar Mena¹, Donggang Jia¹, Alexandre Kouyoumdjian¹, Ivan Viola¹

¹King Abdullah University of Science and Technology, Thuwal, Saudi Arabia.

²University of Electronic Science and Technology of China, Chengdu, China.

ABSTRACT

We evaluate 17 LLMs across four XR devices—Magic Leap 2, Meta Quest 3, Vivo X100s Pro, and Apple Vision Pro—assessing performance on key metrics: consistency, processing speed, and battery consumption. Our experimental setup examines 68 model-device pairs under varying string lengths, batch sizes, and thread counts, providing insights into trade-offs for real-time XR applications. Our findings offer guidance for optimizing LLM deployment on XR devices and establish a foundation for future research in this rapidly evolving field.

Index Terms: Extended reality, large language models, human AI interactions, performance evaluation

1 INTRODUCTION

Since the 2022 release of OpenAI's ChatGPT [5], powered by GPT-3.5, Large Language Models (LLMs) have revolutionized science and technology, showcasing exceptional capabilities across diverse topics, levels of abstraction, and multilingual tasks. Recent advancements in multimodal LLMs enable vision-language integration for tasks such as scene interpretation and software control. However, their reliance on cloud-based processing presents challenges, including latency, privacy concerns, and high costs [5]. Extended reality (XR) and wearable technologies, like Meta AI in VR hardware, also require constant connectivity for inference, limiting their real-world applicability. To address these issues, smaller, on-device LLMs developed by Meta, Mistral AI, Microsoft, and Google [4] offer offline functionality and enhanced privacy. Nonetheless, selecting optimal models and devices remains complex due to varied hardware configurations and LLMs architectures. Establishing standardized evaluation criteria for fair and consistent testing remains an open research challenge.

To address these challenges, we present a comprehensive study deploying LLMs on XR, focusing on performance evaluation across various metrics. Considering the multiple challenges and factors affecting the evaluation, we first identify key evaluation metrics, and then design an experimental setup to minimize potential bias. Specifically, we deploy 17 LLMs across four XR devices—Magic Leap 2, Meta Quest 3, Vivo X100s Pro,¹ and Apple Vision Pro—and conduct a thorough evaluation in terms of processing speed, consistency over time, and battery consumption. This results in 68 model-device pairs. A key challenge is the effect of varying string lengths, batch sizes, thread counts, and background processes. To account for string length variation, we conducted experiments using different lengths. To mitigate hidden factors like background processes, each experiment was repeated five times, and average values were calculated. To provide task-specific analysis, each experiment

was repeated for all LLMs tasks: *Prompt Processing (PP)*, *Token Generation (TG)*, *Batch Test (BT)*, and *Thread Test (TT)*. We believe our experimental protocol and results will serve as a foundation for further research and development in this emerging field.

2 METHODOLOGY

Problem Statement: In this study, we deploy 17 LLMs, denoted $\{m_1, m_2, \dots, m_{17}\}$ (see Tab. 1), across four XR and mobile devices: d_1, d_2, d_3 , and d_4 , representing Magic Leap 2,² Meta Quest 3,³ Vivo X100s Pro,⁴ and Apple Vision Pro,⁵ respectively. Our goal is to perform a comprehensive evaluation of these 68 model-device pairs. We developed a standard evaluation approach, performed various tests, and analyzed results from multiple perspectives.

LLM Deployment on XR Devices: We deployed LLMs locally on XR devices by customizing the Llama.cpp library [3] for four target XR devices. The resulting application supports loading various appropriately sized GPT-Generated Unified Format (GGUF) models and leverages *Llama* functionalities to perform basic LLMs tasks while adjusting parameters. For example, we can control parameters like prompt length in *Prompt Processing (PP)* and token set size in *Token Generation (TG)* to evaluate different aspects in our experiments. Due to compatibility constraints with Llama.cpp, analysis for Apple Vision Pro was conducted on both CPU and GPU, while the remaining three devices were analyzed exclusively on CPU.

Evaluation Approach: Considering the key challenges, we formulate a standardized evaluation approach for our assessments. First, we designed an experimental setup to minimize potential biases and ensure the validity of the results. This setup sets identical parameters across all devices and defines the test types to be executed. We used 17 LLMs (see Tab. 1) in our evaluations. The models were obtained from Hugging Face's model repository, a widely used platform for accessing pre-trained LLMs [2]. We assigned unique IDs (m_1 to m_{17}) to these models.

Considering the *Llama* documentation [1, 3] and our preliminary assessment, this evaluation comprises the following tests.

- **Performance Consistency:** Tests each model-device pair 20 times to assess consistency over time, analyzing mean, standard deviation, and coefficient of variance.
- **Prompt Processing (PP):** Measures device speed in handling input prompts of lengths 64, 128, 256, 512, and 1024.
- **Token Generation (TG):** Assesses device speed in generating output tokens, repeated for token set sizes 64, 128, 256, 512, and 1024.
- **Batch Test (BT):** Evaluates the device's capability to handle parallelization with batch sizes of 64, 128, 256, 512, and 1024.
- **Thread Test:** Measures performance scalability by varying thread counts (1, 2, 4, 8, 16, 32).
- **Battery Usage Test:** Assesses power consumption.

²<https://www.magicleap.com/magicleap-2>

³<https://www.meta.com/quest>

⁴<https://www.vivo.com/en>

⁵<https://www.apple.com/vision-pro>

*Corresponding author: dawar.khan@kaust.edu.sa

¹ Vivo X100s Pro is not an XR device but is included for its strong hardware, as smartphones are generally used in XR applications.

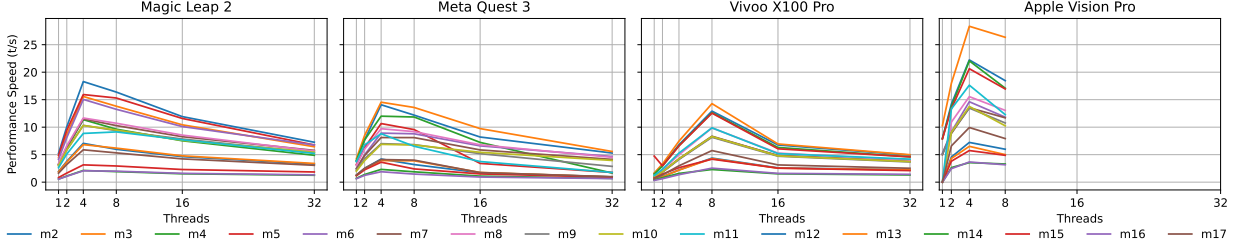


Figure 1: Thread Test (TT) results: The X-axis represents the thread count, while the Y-axis shows the speed in tokens per second. Threads 4 and 8 deliver the fastest results. Note: Apple Vision Pro fails for thread counts of 16 and 32.

Table 1: PP and TG Test Results: Performance speed (t/s) for 17 models across four XR devices, presented as mean (μ) $\pm$ standard deviation (σ) and coefficient of variation (CV). Each value is averaged over five experiments with string/prompt lengths of 64, 128, 256, 512, and 1024.

ID (Size)	Model Name	PP Test ($\mu \pm \sigma$ CV %)				TG Test ($\mu \pm \sigma$ CV %)			
		Magic Leap 2	Meta Quest 3	Vivo X100s Pro	Apple Vis. Pro	Magic Leap 2	Meta Quest 3	Vivo X100s Pro	Apple Vis. Pro
m ₁ (0.942 GB)	qwen2-0.5b-instruct-1p16	66.45 $\pm$ 2.97 4.47	108.95 $\pm$ 13.74 12.61	101.07 $\pm$ 5.87 5.81	294.50 $\pm$ 23.22 7.88	20.21 $\pm$ 0.25 1.24	19.83 $\pm$ 3.97 20.01	22.50 $\pm$ 0.55 2.44	46.35 $\pm$ 1.61 3.47
m ₂ (1.36 GB)	Vikhr-Gemma-2B-instruct-Q3_K_M	24.04 $\pm$ 1.24 5.16	23.30 $\pm$ 3.15 13.52	16.06 $\pm$ 0.52 3.24	32.26 $\pm$ 2.43 7.55	11.04 $\pm$ 0.33 2.99	9.91 $\pm$ 2.89 29.18	8.97 $\pm$ 0.48 5.35	16.52 $\pm$ 4.40 26.61
m ₃ (1.51 GB)	Vikhr-Gemma-2B-instruct-Q4_0	20.04 $\pm$ 0.90 4.49	24.38 $\pm$ 3.61 14.81	17.83 $\pm$ 0.69 3.87	43.61 $\pm$ 4.36 10.00	10.11 $\pm$ 0.26 2.57	10.15 $\pm$ 2.75 27.07	9.73 $\pm$ 0.60 6.16	21.07 $\pm$ 1.50 7.11
m ₄ (1.75 GB)	Vikhr-Gemma-2B-instruct-Q5_0	13.92 $\pm$ 0.42 3.02	18.75 $\pm$ 4.05 21.60	15.64 $\pm$ 0.50 3.20	32.66 $\pm$ 3.22 9.85	8.28 $\pm$ 0.19 2.29	8.46 $\pm$ 2.73 32.26	8.82 $\pm$ 0.51 5.78	15.65 $\pm$ 3.81 24.32
m ₅ (2.00 GB)	Vikhr-Gemma-2B-instruct-Q6_K	23.76 $\pm$ 1.21 5.09	17.34 $\pm$ 3.11 17.94	15.56 $\pm$ 0.51 3.28	33.67 $\pm$ 2.86 8.49	8.34 $\pm$ 0.19 2.28	6.96 $\pm$ 1.83 26.28	8.64 $\pm$ 0.51 5.90	15.62 $\pm$ 4.18 26.76
m ₆ (1.32 GB)	Phi-3.1-mini-4k-instruct-Q2_K	16.74 $\pm$ 0.83 4.96	11.37 $\pm$ 2.68 23.58	9.00 $\pm$ 0.22 2.44	19.59 $\pm$ 1.67 8.52	11.70 $\pm$ 0.73 6.24	6.91 $\pm$ 2.18 31.50	6.94 $\pm$ 0.46 6.63	14.03 $\pm$ 1.53 10.90
m ₇ (1.94 GB)	Phi-3.1-mini-4k-instruct-Q3_K_L	13.52 $\pm$ 0.55 4.07	10.49 $\pm$ 2.69 25.64	9.17 $\pm$ 0.22 2.40	18.49 $\pm$ 2.55 13.81	8.54 $\pm$ 0.39 4.57	6.74 $\pm$ 2.11 31.31	6.98 $\pm$ 0.53 7.59	12.73 $\pm$ 1.73 13.62
m ₈ (2.30 GB)	Phi-3.1-mini-4k-instruct-Q4_K_L	14.48 $\pm$ 0.62 4.28	12.83 $\pm$ 3.30 25.71	10.60 $\pm$ 0.29 2.74	22.68 $\pm$ 2.12 9.33	7.96 $\pm$ 0.34 4.27	6.73 $\pm$ 1.94 28.78	8.02 $\pm$ 0.70 8.73	15.77 $\pm$ 1.40 8.85
m ₉ (2.68 GB)	Phi-3.1-mini-4k-instruct-Q5_K_L	12.99 $\pm$ 0.50 3.85	9.05 $\pm$ 2.38 26.30	8.72 $\pm$ 0.44 5.05	18.61 $\pm$ 1.81 9.75	6.91 $\pm$ 0.26 3.76	5.53 $\pm$ 1.83 33.02	6.43 $\pm$ 0.86 13.37	11.93 $\pm$ 2.96 24.81
m ₁₀ (2.92 GB)	Phi-3.1-mini-4k-instruct-Q6_K	13.65 $\pm$ 0.54 3.96	9.15 $\pm$ 2.05 22.40	9.99 $\pm$ 2.06 20.62	18.21 $\pm$ 1.60 8.80	6.28 $\pm$ 0.22 3.50	5.96 $\pm$ 1.95 32.74	6.72 $\pm$ 0.94 13.99	12.45 $\pm$ 1.52 12.23
m ₁₁ (3.78 GB)	Phi-3.1-mini-4k-instruct-Q8_0	12.06 $\pm$ 0.45 3.73	13.52 $\pm$ 3.88 28.71	11.45 $\pm$ 0.71 6.20	27.64 $\pm$ 3.26 11.79	5.04 $\pm$ 0.13 2.58	4.76 $\pm$ 1.38 28.91	6.90 $\pm$ 1.04 15.07	12.09 $\pm$ 0.25 2.10
m ₁₂ (2.63 GB)	llama-2-7b-chat-Q2_K	7.89 $\pm$ 0.24 3.04	5.13 $\pm$ 1.00 19.50	4.78 $\pm$ 0.06 1.26	9.09 $\pm$ 0.63 6.98	5.89 $\pm$ 0.26 4.41	3.39 $\pm$ 1.06 31.25	3.70 $\pm$ 0.18 4.86	6.24 $\pm$ 1.85 29.65
m ₁₃ (2.75 GB)	llama-2-7b-chat-Q3_K_S	7.49 $\pm$ 0.22 2.94	4.81 $\pm$ 1.16 24.11	4.66 $\pm$ 0.15 3.22	8.72 $\pm$ 0.59 6.75	5.69 $\pm$ 0.24 4.22	2.90 $\pm$ 0.96 33.23	3.60 $\pm$ 0.22 6.11	6.36 $\pm$ 1.56 24.57
m ₁₄ (1.64 GB)	Mistral-7B-Instruct-v0.3.IQ1_M	2.23 $\pm$ 0.02 0.90	2.67 $\pm$ 0.62 23.22	2.54 $\pm$ 0.03 1.18	5.38 $\pm$ 0.26 4.86	2.07 $\pm$ 0.01 0.48	2.07 $\pm$ 0.61 29.25	2.01 $\pm$ 0.06 2.99	4.50 $\pm$ 0.29 6.36
m ₁₅ (2.05 GB)	Mistral-7B-Instruct-v0.3.IQ2_XS	3.25 $\pm$ 0.04 1.23	4.21 $\pm$ 1.05 24.94	4.53 $\pm$ 0.14 3.09	7.34 $\pm$ 0.86 11.71	2.97 $\pm$ 0.03 1.01	2.81 $\pm$ 0.86 30.54	3.55 $\pm$ 0.20 5.63	5.95 $\pm$ 0.55 9.19
m ₁₆ (2.81 GB)	Mistral-7B-Instruct-v0.3.IQ3_XS	2.15 $\pm$ 0.02 0.93	2.20 $\pm$ 0.38 17.27	2.79 $\pm$ 0.02 0.72	4.81 $\pm$ 0.33 6.95	1.98 $\pm$ 0.01 0.51	1.55 $\pm$ 0.27 17.37	2.24 $\pm$ 0.11 4.91	5.43 $\pm$ 1.61 29.72
m ₁₇ (3.64 GB)	Mistral-7B-Instruct-v0.3.IQ4_XS	6.68 $\pm$ 0.16 2.40	5.13 $\pm$ 1.28 24.95	6.30 $\pm$ 0.22 3.49	13.54 $\pm$ 1.00 7.41	4.84 $\pm$ 0.08 1.65	2.89 $\pm$ 0.95 32.92	4.69 $\pm$ 0.32 6.82	9.04 $\pm$ 2.43 26.91

3 RESULTS

Tab. 1 provides quantitative results for the Prompt Processing (PP) and Token Generation (TG) tests. For each string/prompt length (64, 128, 256, 512, and 1024), we ran five experiments and recorded the mean (μ), standard deviation (σ), and coefficient of variation (CV). Instead of reporting individual results for each string length, we calculated and reported the average of these measurements across all string lengths.

Fig. 1 illustrates the results of the Thread Test (TT), where higher speeds are observed for thread counts of 4 and 8. Fig. 2 presents consistency results, showing that the Vision Pro achieves the fastest results and most consistent performance. However, we encountered multiple failures of the Vision Pro during the experiments. In the BT results, a batch size of 128 on Apple Vision Pro and Vivo performed better, with no significant differences observed in the other cases (see Fig. 3). Battery consumption during the 10-minute experiments was 12.8% for Vision Pro, 10.2% for Meta Quest 3, 8.0% for Magic Leap 2, and 4.2% for Vivo X100s Pro.

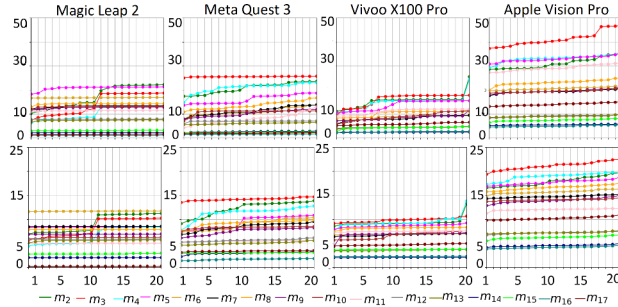


Figure 2: Consistency results of the four devices over time: PP (top) and TG (bottom) speeds in tokens per second across 20 sorted runs (X-axis: run number, Y-axis: speed in t/s).

4 CONCLUSION

This study deploys LLMs on four devices and conducts a comprehensive evaluation using a fair experimental setup. This work sets

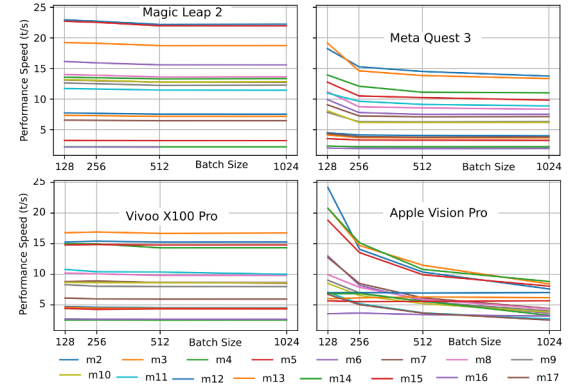


Figure 3: Batch Test results: Batch size (X-axis) vs. speed (Y-axis).

a baseline for on-device AI in XR, helping researchers benchmark models, devices, and algorithms. To our knowledge, this study is the first of its kind, pioneering the evaluation of LLMs locally in XR environments.

REFERENCES

- [1] AI-Meta. Introducing llama: A foundational, 65-billion-parameter large language model. *Meta AI*, 2023. 1
- [2] H. Face. Hugging Face: The AI community building the future. www.huggingface.co, 2024. (Accessed: Sep. 10, 2024). 1
- [3] G. Gerganov. llama.cpp: LLM inference in C/C++. <https://github.com/ggerganov/llama.cpp>, 2023. (Accessed: Jan. 2, 2025). 1
- [4] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *Proceedings of ICML*, pp. 32431–32454, 2024. doi: 10.48550/arXiv.2402.14905 1
- [5] T. Wu, S. He, J. Liu, S. Sun, K. Liu, Q.-L. Han, and Y. Tang. A brief overview of ChatGPT: The history, status quo and potential future development. *IEEE/CAA Journal of Automatica Sinica*, 10(5):1122–1136, 2023. doi: 10.1109/JAS.2023.123618 1